

The Observer Design Pattern

Selection Context

The system requires a mechanism where multiple user interface (UI) components (such as the statistics dashboard, the patient data table, the staff data table, the departmental tree, and the real-time event log) constantly reflect the most recent state of the hospital's central database (HospitalData).

Justification & Advantages

- **Elimination of Tight Coupling:** Without the Observer pattern, HospitalData would need direct references to DashboardObserver, PatientTableObserver, StaffTableObserver, etc., to explicitly call their refresh methods. This would tightly couple the core business logic to the UI framework (Java Swing). By introducing the HospitalObserver interface, HospitalData remains completely blind to the UI implementation details.
- **Dynamic Subscriptions (Runtime Flexibility):** Views can dynamically attach or detach themselves at runtime using the subscribe() and unsubscribe() methods. For instance, if a user closes the "Event Log Panel," that specific observer can call unsubscribe(), instantly stopping unnecessary update broadcasts and saving CPU cycles.
- **Open/Closed Principle (OCP) Compliance:** If the hospital system expands and requires a new visualization view (e.g., a graphical Chart View or an automated SMS Alert System), a new observer class can be created to implement HospitalObserver. This new feature can be plugged into the notification stream without modifying a single line of code in the existing HospitalData subject class.